

An Efficient Verification Approach to Separation of Duty in Attribute-Based Access Control

Benyuan Yang  and Hesuan Hu , *Senior Member, IEEE*

Abstract—The problem considered in this paper is the verification and enforcement of separation of duty (SoD) constraints in attribute based access control (ABAC) systems. We propose an efficient algorithm for checking the satisfiability of SoD constraints. It is based on the idea of partitioning all permissions of SoD constraints into two classes so as to compute the minimal number of users to accomplish each class of permissions, respectively. As a result, several SoD constraints with certain class of permissions can be verified in polynomial time. Experimental results show that our method performs well compared with existing ones. When SoD violations occur, a 0–1 integer programming (IP) based enforcement solution is presented such that SoD violations can be solved once for all and it is provably shown that such solution does not result in the violation of other SoD constraints.

Index Terms—Attribute based access control, polynomial-time verification, separation of duty, violation resolving.

I. INTRODUCTION

ACCESS control is a mechanism of restricting access to sensitive data [7], [26]. It is useful for minimizing the risk of authorized access to physical and computer systems, thus forming a foundational part of system security [3], [27], [29], [30], [31], [32]. The main types of access control used nowadays are: role-based access control (RBAC) [25] and attribute-based access control (ABAC) [11].

Verification and enforcement are two important issues in the study of access control systems [16]. For large and complex access control systems, we are first interested in verifying whether or not the given system satisfies a certain authorization constraint, e.g., separation of duty (SoD) [2], [4]; when the answer

is negative, we wish to obtain a solution so as to enforce all SoD constraints, i.e., making all SoD violations satisfied while not influencing the satisfaction of other SoD constraints [5], [34].

In the context of RBAC, many approaches have been proposed for the verification and enforcement of SoD constraints. In [1], SoD constraints are modeled as formulae in linear temporal logic such that they can be formally verified by model checking. A novel algebra is introduced in [20] to model and verify high-level SoD constraints so as to capture not only qualitative requirements but also quantitative ones on users. Distributed verification policy is proposed in [33] such that SoD constraints can be verified through local control without knowing global information. The authors in [21] show that directly checking the satisfiability of SoD constraints in RBAC is coNP-complete, while determining the satisfaction of mutually exclusive role (MER) constraints can be done in P. As a result, SoD constraints can be verified efficiently after translating SoD constraints to MER constraints. However, it is also coNP-complete to verify whether a given set of MER constraints are adequate to enforce an SoD constraint. In [10], a constraint-satisfied system is constructed by adding controllers such that SoD constraints are directly enforced in the system. Integer programming (IP) based approaches are discussed in [28] and [35], where the satisfaction of SoD constraints is ensured by certain IP constraints. If an optimal solution is returned by the IP, then no constraint is violated.

While there is a wide literature on the verification and enforcement of SoD constraints in RBAC systems, such issue has not received much attention in ABAC systems. Nevertheless, it is important to impose SoD constraints on ABAC systems. For example, there was an access control loophole in KFC's mobile application and WeChat account due to the existence of SoD violations. As a consequence, people can swindle the online ordering system for KFC meals and coupons without actually paying for them (see <https://www.globaltimes.cn/page/202105/1223171.shtml>). Clearly, without proper verification and enforcement, an SoD violation can lead to the fact that prohibited access may become accessible, thus leading to unexpected disasters. Hence, it is of great significance to verify and enforce SoD constraints so as to ensure the access control security of such systems as well as reduce the costs and risks caused by SoD violations.

The problem of SoD verification in ABAC is partly solved in the existing literature. The methods proposed in [17], [22], [36], [37] conduct in a direct way to check the satisfiability of SoD constraints by computing the maximal number of users

Manuscript received 21 October 2022; revised 13 April 2023; accepted 1 March 2024. Date of publication 5 March 2024; date of current version 7 August 2024. This work was supported in part by the Natural Science Foundation of China under Grant 61973242, Grant 61573265, and Grant 61203037, in part by the Fundamental Research Funds for the Central Universities under Grant K7215581201, Grant K5051304004, and Grant K5051304021, in part by New Century Excellent Talents in University under Grant NCET-12-0921, in part by the Academic Research Fund Tier 1 by Ministry of Education in Singapore under Grant 2014-T1-001-147, in part by the Academic Research Fund Tier 2 by Ministry of Education in Singapore under Grant MOE2015-T2-2-049, and in part by the Major Fundamental Research Program of the Natural Science Foundation of Shaanxi Province under Grant 2017ZDJC-34. Recommended for acceptance by Y. Gao. (Corresponding author: Hesuan Hu.)

Benyuan Yang is with the School of Future Science and Engineering, Soochow University, Suzhou 215299, China (e-mail: byyang@suda.edu.cn).

Hesuan Hu is with the School of Electro-Mechanical Engineering, Xidian University, Xi'an 710071, China, also with the School of Computer Science and Engineering, College of Engineering, Nanyang Technological University, Singapore 639798, and also with the State Key Laboratory for Manufacturing Systems Engineering, Xi'an Jiaotong University, Xi'an 710054, China (e-mail: huhesuan@gmail.com).

Digital Object Identifier 10.1109/TKDE.2024.3373562

that can finish all permissions in each SoD constraint. However, the above methods suffer from computational difficulties. An improved SoD verification method for ABAC is proposed in [14]. Motivated by [21], MERs are defined and used in [14]. It is shown that directly checking SoD constraints in an ABAC system is coNP-complete, while checking whether the ABAC system satisfies a set of MER constraints can be done in P. Thus, MERs are used as an alternative to check the satisfiability of SoD constraints. However, MERs can handle only two special types of SoD constraints, i.e., the number of users restricted in the SoD constraints is equal to two or the number of permissions in the SoD constraints. This in turn implies that MERs cannot be used as a general-purpose verification mechanism to check the satisfiability of all SoD constraints in real-world scenarios. Furthermore, although MER constraints can be verified efficiently, checking the satisfaction of an SoD constraints through MER constraints needs to generate separation of rules at first. Such a process is shown to be #P-complete (a problem is #P-complete if and only if it is in #P, and for any non-deterministic Turing machine, the problem of computing its number of accepting paths can be reduced to this problem). This means that MER-based method is practically infeasible. When some SoD constraints are not satisfied, it can lead to fraud, errors, and other risks in the ABAC. Hence, it is critical to find a solution that can correct all SoD violations so as to guarantee the authorization security of ABAC. However, this issue is not fully investigated in existing literature [14], [17], [22], [36], [37].

In this paper, we present an efficient verification method for checking the satisfaction of SoD constraints in ABAC systems, and a 0–1 IP-based method to solve SoD violations, if exist. The approach and contributions of this paper are briefly summarized as follows:

- We classify SoD constraints into several types by subdividing permissions of SoD constraints into two classes. It is proved that the satisfiability of certain types of SoD constraints can be checked in polynomial time. This means the satisfaction of SoD constraints can be determined in an efficient way. Compared with existing methods [17], [22], [36], [37], the performance of our approach is extremely improved. Furthermore, our approach can be applied to all SoDs. It is feasible in real-world scenarios compared with the method in [14].
- We present a 0–1 IP-based method to solve SoD violations. It is conducted by forbidding the minimal number of access relations between certain users and rules so as to ensure that the minimum number of users who together complete all permissions in each SoD constraint is greater than or equal to a required number. All SoD violations can be corrected in a once for all manner and all SoD constraints are satisfied after correction. This significantly improves the authorization security of ABAC systems.

The application areas of our approach include, but are not limited to, intelligent transportation systems [8], NoSQL database systems [9], and enterprise management systems [23], [24], to which ABAC is successfully applicable. An SoD constraint prevents two or more subjects from accessing an object that lies within their conflict of interests or disallows a subject

from accessing conflicting objects or permissions. For industrial intelligent transportation systems, NoSQL database systems, and enterprise management systems, there exist many direct or indirect constraints that can be expressed exactly by SoD constraints. For instance, the drivers of two or more smart entities should not broadcast the same notifications to the other participating smart entities, the contents in a set of columns are only applicable to several high-level users, and a same manager cannot authorize payments as well as sign the payment checks.

Our approach can be directly used to guide the SoD constraint verification and violation resolution in these systems. On one hand, different requirements, being defined as SoD constraints, can be properly captured by our method since it is adequate to check all SoD constraints. On the other hand, our method can verify SoD constraints in an efficient way. As more users, rules, and permissions are involved in the concerned systems, SoD constraint verification becomes a challenging problem. Based on our approach, we can ensure that certain types of SoD constraints can be checked in polynomial time while the others can be verified within a reasonable time. Furthermore, we present an effective violation resolution method. This not only guarantees the access control security of the above practical systems but also increases the users' confidence in accessing permissions.

The remainder of this paper is organized as follows. The summary and analysis of related works are given in Section II. Section III presents the preliminaries of ABAC and SoD. Section IV formulates the problem to solve in this paper. Sections V and VI show the detailed procedures to check SoD constraints and solve SoD violations, if exist, respectively. The experimental results are given in Section VII. Section VIII concludes this paper and outlines future work.

II. RELATED WORK

In recent years, ABAC has been successfully applicable to different scenarios. The authors in [8] apply ABAC to cloud-assisted intelligent transportation system so as to enable location-specific and instant notifications or alerts in smart transportation environment. The authors in [9] present a novel approach to integrating a full-fledged support for ABAC at a fine granularity into NoSQL databases. A method is proposed in [19] for automatically learning ABAC policy rules from access logs of a system to simplify the policy development process. The authors in [18] present a novel approach to improving the authorization of the DDS (i.e., middleware standard for data-centric publish/subscribe communication in a large scale real-time environment) security model using ABAC. A novel distributed, dynamic, and trusted access control model is given in [12]. It is modified and extended from ABAC, and can be better applied to the distributed environment. The authors in [6] propose SmartABAC: a fast, concise, and expressive ABAC model that can be executed in constrained IoT devices. However, the verification and enforcement of SoD constraints are not considered in these works.

As we discussed above, dealing with the satisfaction checking problem of SoD constraints and solving the problem of SoD violations in ABAC systems are extremely necessary.

The problem of SoD verification in ABAC is partly solved in existing literature. The authors in [17] present a policy specification language to specify SoDs on attribute assignment. An attribute based constraint specification language (ABCL) is presented in [36] to represent different kinds of SoDs among attributes. ABCL ensures that the constraints are event independent and are to be uniformly enforced no matter what event has caused an attribute value to change. Enforcement of ABCL constraints is discussed in [37], and it has been shown to be scalable. The authors in [22] examine potential relationships within and across the sets of subjects and objects. However, the above methods suffer from computational difficulties, which render them impracticable. Furthermore, the solution to violated SoDs is not considered in these works.

The authors in [15] show that while SoD verification is intractable, a part of it can be efficiently solved using the principle of mutual exclusion. It is then extended by [14] so as to comprehensively address the problem of efficient generation of MER constraints corresponding to a given SoD constraint. The method in [14] is further used in [23] and [24] for solving employee replacement problem and obtaining optimal employee recruitment in ABAC, respectively. However, the above methods can handle only two special types of SoD constraints, i.e., the number of users restricted in the SoD constraints is equal to two or the number of permissions in the SoD constraints. Furthermore, they do not consider how to solve SoD violations.

Therefore, this paper focuses on how to efficiently verify SoD constraints and effectively solve SoD violations, if exist.

III. PRELIMINARIES

A. Attribute Based Access Control (ABAC) Model

Definition 1: The elements of ABAC are defined as follows:

- (Users $\mathcal{U}$ and User Attributes $\mathcal{UA}$) A user is denoted as $u \in \mathcal{U}$, and is associated with several user attributes (such as name, age, and affiliation). We represent the name (un) and value (uv) of each attribute by a pair *name-value* in the form of $(un = uv) \in \mathcal{UA}$. Thus, $\mathcal{UA}(u)$ can be used to represent the set of attributes for user u .
- (Objects $\mathcal{O}$ and Object Attributes $\mathcal{OA}$) An object (such as file, device, and program) is denoted as $o \in \mathcal{O}$, and is characterized by several object attributes. Each object attribute is specified by a *name-value* pair $(on = ov) \in \mathcal{OA}$. The set of attributes for object o is denoted as $\mathcal{OA}(o)$.
- (Actions $\mathcal{A}$ (interchangeably called operations)) An action (such as read and write) is denoted as $a \in \mathcal{A}$, which illustrates the operation a user can perform on an object.
- (Rules $\mathcal{R}$) A rule is denoted as $r \in \mathcal{R}$, which includes a set of user attributes, a set of object attributes, and an action. Formally, $r = \{UC_r, OC_r, a_r\}$, where UC_r (resp., OC_r) represents the set of user attributes (resp., object attributes), each of which is of the form $(un = uv)$ (resp., $(on = ov)$). Note that, the value of a user attribute uv (resp., the value of an object attribute ov) can be “any” (by the term “any”, we mean that the value of the corresponding attribute can be any feasible value). Action $a_r \in \mathcal{A}$ is an operation that can be performed by a user on an object.

TABLE I
USERS AND USER ATTRIBUTES

<i>user index</i>	<i>user</i>	<i>department</i>	<i>role</i>	<i>experience</i>
u_1	Alice	sales	executive	10
u_2	Bob	sales	manager	5
u_3	Charles	sales	executive	10
u_4	Davis	sales	teamleader	3
u_5	Emily	sales	teamleader	8

TABLE II
OBJECTS AND OBJECT ATTRIBUTES

<i>object</i>	<i>type</i>	<i>content</i>	<i>area</i>
File1	document	sales_plan	east coast
File2	document	sales_report	east coast
File3	contract	sales_contract	west coast
File4	contract	sales_contract	east coast

In the context of ABAC, users and objects are uniquely characterized by well-defined attributes. Rules are specified through the combination of user and object attributes as well as operations. In ABAC, a user can perform an operation upon a target object if there exists a rule that authorizes the user to do so. Specifically, user $u \in \mathcal{U}$ can conduct operation $a \in \mathcal{A}$ on object $o \in \mathcal{O}$ if the user attributes of u , object attributes of o , and action a exactly match with UC_r , OC_r , and a_r , respectively, specified in a rule $r \in \mathcal{R}$. Note that, a set of environment conditions and attributes are defined in standard ABAC model [11]. However, we omit them in this paper since the considered SoD constraints (will be presented in the sequel) are independent of environment.

Example 1: Consider an ABAC example of users accessing a file management system, which is excerpted and adapted from [14]. The example includes the following users, objects, and actions.

- $\mathcal{U}$: {Alice, Bob, Charles, Davis, Emily}.
- $\mathcal{O}$: {File1, File2, File3, File4}.
- $\mathcal{A}$: {Prepare, Design, Review, Approve}.

The attributes of users and objects are given in Tables I and II, respectively. The rules are defined in Table III.

B. Separation of Duty (SoD) Constraints in ABAC

SoD constraints are widely implemented in different access control systems so as to ensure authorization security. Usually, an SoD constraint defines that a set of permissions should be performed by at least a given number of users. Here, permissions essentially imply authority to perform operations on certain types of objects. In ABAC system, a permission can be formally defined as follows.

Definition 2: A permission p can be expressed as a pair (a, o) , where action $a \in \mathcal{A}$ and object $o \in \mathcal{O}$. The set of all permissions in an ABAC system is denoted as $\mathcal{P}$.

Using the notion of permissions, we can define SoD constraints in ABAC as follows. Let $\mathcal{S}$ be the set of SoD constraints. Given $K \in \mathbb{Z}^+ = \{1, 2, \dots\}$, we use $\mathbb{Z}_K = \{1, 2, \dots, K\}$ to represent the set of integers from 1 to K .

Definition 3: The i th SoD constraint in an ABAC model is expressed as $\mathcal{S}_i = \langle \{p_1, p_2, \dots, p_n\}, \Diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$, $n \in \mathbb{Z}_{|\mathcal{P}|}$, and $\Diamond_i \in \mathbb{Z}_{|\{p_1, p_2, \dots, p_n\}|}$.

TABLE III
RULES

$r_1 = \{(department = sales), (role = executive), (experience = 10)\}, \{(type = document), (content = any), (area = any)\}, Approve\}$
$r_2 = \{(department = sales), (role = any), (experience = 10)\}, \{(type = any), (content = any), (area = east coast)\}, Approve\}$
$r_3 = \{(department = sales), (role = manager), (experience = 5)\}, \{(type = document), (content = any), (area = any)\}, Review\}$
$r_4 = \{(department = sales), (role = any), (experience = 10)\}, \{(type = document), (content = any), (area = east coast)\}, Design\}$
$r_5 = \{(department = sales), (role = any), (experience = 10)\}, \{(type = document), (content = any), (area = east coast)\}, Design\}$
$r_6 = \{(department = sales), (role = teamleader), (experience = 3)\}, \{(type = document), (content = sales_plan), (area = any)\}, Design\}$
$r_7 = \{(department = sales), (role = manager), (experience = 5)\}, \{(type = document), (content = sales_report), (area = any)\}, Prepare\}$
$r_8 = \{(department = sales), (role = any), (experience = 8)\}, \{(type = contract), (content = sales_contract), (area = any)\}, Approve\}$

An SoD constraint $\mathcal{S}_i$ requires that the minimum number of users together to finish all permissions in $\mathcal{S}_i$ should be greater than or equal to $\diamond_i$. In other words, SoD constraint $\mathcal{S}_i$ is violated if there exists a user set with size $\diamond_i - 1$ or less number of users that can perform all permissions in the SoD constraint. Unless otherwise specified, from now on we use $\mathcal{P}_i = \{p_1, p_2, \dots, p_n\}$ to represent the set of permissions involved in SoD constraint $\mathcal{S}_i$. As a result, SoD constraint $\mathcal{S}_i$ can be equivalently written as $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$.

Three SoD constraints are specified for the ABAC in Example 1 as follows: $\mathcal{S} = \{\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3\}$, where

- $\mathcal{S}_1 = \langle \{(Prepare, File2), (Review, File2), (Approve, File2)\}, 2 \rangle$.
- $\mathcal{S}_2 = \langle \{(Design, File1), (Review, File1), (Approve, File1)\}, 3 \rangle$.
- $\mathcal{S}_3 = \langle \{(Approve, File1), (Approve, File3), (Approve, File4), (Review, File1)\}, 3 \rangle$.

IV. THE PROBLEM OF SOD VERIFICATION IN ABAC

In ABAC, a user can have access to a permission when there exists at least one rule which allows the user to do so. Therefore, according to Definition 3 we can determine the satisfiability of an SoD constraint by computing different sets of users, each of which can complete all permissions in the SoD constraint by accessing certain rules that possess all permissions in the SoD constraint. If there exists a user set that contains less than $\diamond_i$ users, then we say that the considered SoD constraint $\mathcal{S}_i$ is violated. Otherwise, it is satisfied.

To establish a relation between users, rules, and permissions, we define the following sets, namely, valid rule set of user and relevant rule set of permission.

Definition 4: Consider a user $u \in \mathcal{U}$, an object $o \in \mathcal{O}$, and a rule $r = \{UC_r, OC_r, a_r\} \in \mathcal{R}$. It holds that 1) $\mathcal{UA}(u) = UC_r$ if $\forall(un = uv) \in \mathcal{UA}(u)$, $\exists(un = uv)$ or $(un = any) \in UC_r$; and that 2) $\mathcal{OA}(o) = OC_r$ if $\forall(on = ov) \in \mathcal{OA}(o)$, $\exists(on = ov)$ or $(on = any) \in OC_r$.

Definition 5: Consider an ABAC model and a user $u \in \mathcal{U}$. The valid rule set of user u is a set of rules in which each rule in this set allows u to perform the permission given in the rule. Such a set can be formally defined as

$$\Theta(u) = \{r \in \mathcal{R} \mid \mathcal{UA}(u) = UC_r\}.$$

Definition 6: Consider an ABAC model and the i th SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. The relevant rule set for a single permission $p_k = (a, o) \in \mathcal{P}_i$ is a set of rules, each of which allows some users to access p_k , where $k \in \mathbb{Z}_{|\mathcal{P}_i|}$, $a \in \mathcal{A}$, and $o \in \mathcal{O}$. We can formally express such a

TABLE IV
VALID RULE SET FOR EACH USER IN EXAMPLE 1

user	rule
u_1	$\{r_1, r_2, r_4, r_5\}$
u_2	$\{r_3, r_7\}$
u_3	$\{r_1, r_2, r_4, r_5\}$
u_4	$\{r_6\}$
u_5	$\{r_8\}$

TABLE V
RELEVANT RULE SET FOR EACH PERMISSION IN SOD CONSTRAINT $\mathcal{S}_1$

permission	rule
p_1	$\{r_7\}$
p_2	$\{r_3\}$
p_3	$\{r_1, r_2\}$

TABLE VI
RELEVANT RULE SET FOR EACH PERMISSION IN SOD CONSTRAINT $\mathcal{S}_2$

permission	rule
p_1	$\{r_4, r_5, r_6\}$
p_2	$\{r_3\}$
p_3	$\{r_1, r_2\}$

set as

$$\Psi(p_k) = \{r \in \mathcal{R} \mid \mathcal{OA}(o) = OC_r \wedge a = a_r\}.$$

Using the above defined two sets, the set of all permissions in SoD constraint $\mathcal{S}_i$ to which a user u can be authorized is represented as:

$$\omega_i(u) = \{p_k \in \mathcal{P}_i \mid \Psi(p_k) \cap \Theta(u) \neq \emptyset\}.$$

Accordingly, the set of all permissions in SoD constraint $\mathcal{S}_i$ to which a set of users U is authorized is denoted as:

$$\Omega_i(U) = \bigcup_{u \in U} \omega_i(u).$$

To verify whether an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$ is satisfied or not, a simple and direct method is to compute $\Omega_i(U')$ for each subset $U' \subseteq U$ of users in the ABAC. If there is no subset of users for which both $\mathcal{P}_i \in \Omega_i(U')$ and $|U'| < \diamond_i$ are true, then the given SoD is satisfied. The above verification procedures can be extended to a set of SoD constraints. We denote the above strategy as direct method for checking the satisfiability of SoD constraints (denoted as DSoD-based method in the sequel).

Example 2: Reconsider the ABAC model in Example 1 and three SoD constraints given above. The valid rule set for each user is shown in Table IV. For simplicity, we use user index to represent each user. The relevant rule sets for the permissions in $\mathcal{S}_1, \mathcal{S}_2$, and $\mathcal{S}_3$ are shown in Tables V, VI, and VII, respectively.

TABLE VII
RELEVANT RULE SET FOR EACH PERMISSION IN SoD CONSTRAINT $\mathcal{S}_3$

permission	rule
p_1	$\{r_1, r_2\}$
p_2	$\{r_8\}$
p_3	$\{r_2, r_8\}$
p_4	$\{r_3\}$

For economy of space, we use permission index to represent each permission in the corresponding SoD constraint.

From Tables IV–VII, it can be seen that, both $\mathcal{S}_1$ and $\mathcal{S}_3$ are satisfied, while $\mathcal{S}_2$ is violated. Specifically, at least two users u_1 and u_2 (or u_2 and u_3) can perform all three permissions in $\mathcal{S}_1$ and at least three users u_1, u_2 , and u_5 (or u_2, u_3 , and u_5) together can complete all four permissions in $\mathcal{S}_3$. As for $\mathcal{S}_2$, it requires that at least three different users are needed to finish all three permissions in $\mathcal{S}_2$. A possible set of rules to complete these permissions is r_1, r_3 , and r_4 . From Table IV, it can be easily observed that only two users u_1 and u_2 are enough in quantity to access these three rules, thus leading to the fact that only two users together can finish the three permissions in $\mathcal{S}_2$. Clearly, the number of users actually used (=2) in the ABAC system is less than the number of users required (=3) by constraint $\mathcal{S}_2$. Hence, $\mathcal{S}_2$ is violated.

Now, we can formally define the problem of SoD verification in an ABAC system.

Definition 7: SC-SoD (the satisfiability checking problem for SoD constraints) is defined to be the following problem: Given an ABAC system and a set of SoD constraints $\mathcal{S}$, determine whether each SoD constraint in $\mathcal{S}$ is satisfied.

In [14], it has been shown that SC-SoD is hard, if not impossible, to be solved since it is coNP-complete. While SC-SoD is intractable in general, the authors in [14] state that efficient algorithms exist for SC-SoD when using MERs to enforce SoD constraints. Basically, two rules are declared to be mutually exclusive in the sense that no user is allowed to access both rules. A generalized form of MERs can be formally defined as follows.

Definition 8: An MER constraint is expressed as $\text{MER}\langle\{r_x, r_y, \dots, r_z\}, k\rangle$, where $r_x, r_y, \dots, r_z \in \mathcal{R}$, $x, y, z \in \mathbb{Z}_{|\mathcal{R}|}$, and $k \in \mathbb{Z}_{|\{r_x, r_y, \dots, r_z\}|}$.

The definition of MER means that no user in the ABAC system can access k or more rules in the rule set $\{r_x, r_y, \dots, r_z\}$. Since each MER constraint restricts the rules that cannot be accessed by a single user, it is easy to solve the satisfaction checking problem for MER constraints (denoted as SC-MER), i.e., determining whether a set of MER constraints are satisfied in an ABAC system. The details are as follows: compute the valid rule set of each user in the ABAC system at first, and then compare it with the set of rules involved in the MER constraints.

Theorem 1: SC-MER is in P.

Proof: See Theorem 3 of [14], where an exhaustive proof is given. ■

The fact that SC-SoD is intractable and that SC-MER can be solved in an efficient manner appears to provide a justification for using MER constraints to enforce SoD constraints. Hence, an MER-based method is presented in [14] to check the

satisfiability of SoD constraints. Consider MER constraints are expressed in terms of restrictions on rules. In order to generate MER constraints for verifying SoD constraints, the first step is to translate restrictions on permissions expressed in SoD constraints to restrictions on rules. We define such constraints as separation of rule (SoR) constraints, whose definition is formally given as follows.

Definition 9: Consider an SoD constraint $\mathcal{S}_i = \langle\{p_1, p_2, \dots, p_n\}, \diamond_i\rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$, $n \in \mathbb{Z}_{|\mathcal{P}|}$, and $\diamond_i \in \mathbb{Z}_{|\{p_1, p_2, \dots, p_n\}|}$. Let $m = |\Psi(p_1) \times \Psi(p_2) \times \dots \times \Psi(p_n)|$. The set of SoR constraints corresponding to $\mathcal{S}_i$ can be represented as $\{\text{SoR}_1, \text{SoR}_2, \dots, \text{SoR}_m\}$, where each of them is in the form $\text{SoR}\langle\{r_x, r_y, \dots, r_z\}, \diamond_i\rangle$ satisfying $r_x \in \Psi(p_1)$, $r_y \in \Psi(p_2)$, $\dots$, and $r_z \in \Psi(p_n)$, $r_x, r_y, \dots, r_z \in \mathcal{R}$, and $x, y, \dots, z \in \mathbb{Z}_{|\mathcal{R}|}$.

An SoR constraint restricts that at least $\diamond_i$ users are required together to access all the rules in the SoR constraint.

Then, each SoR constraint is converted into an MER constraint or a set of MER constraints based on the following function (Algorithm 2 of [14]).

Function: Generate MER constraints by an SoR constraint $\text{SoR}\langle\{r_x, r_y, \dots, r_z\}, \diamond_i\rangle$, where $r_x, r_y, \dots, r_z \in \mathcal{R}$, $x, y, \dots, z \in \mathbb{Z}_{|\mathcal{R}|}$, and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}|}$.

if ($\diamond_i == 2$)

return $\text{MER}\langle\{r_1, r_2, \dots, r_n\}, |\mathcal{P}_i|\rangle$;

elseif ($\diamond_i == |\mathcal{P}_i|$)

return $\text{MER}\langle\{r_1, r_2, \dots, r_n\}, 2\rangle$;

else

return a set of MER constraints, each of which is a size- $(\lfloor \frac{|\mathcal{P}_i|}{\diamond_i} \rfloor + 1)$ subsets of $\{r_x, r_y, \dots, r_z\}$ associated with $k = \lfloor \frac{|\mathcal{P}_i|}{\diamond_i} \rfloor + 1$, where $\lfloor \cdot \rfloor$ denotes the largest integer less than or equal to $\cdot$.

Example 3: Consider again the ABAC model in Example 1 and the three SoD constraints $\mathcal{S}_1, \mathcal{S}_2$, and $\mathcal{S}_3$. We present the detailed procedures to check the satisfiability of three SoD constraints using MER-based method.

1) Consider $\mathcal{S}_1 = \langle\{p_1, p_2, p_3\}, 2\rangle$ at first. The set of SoR constraints corresponding to $\mathcal{S}_1$ is: $\text{SoR}\langle\{r_1, r_3, r_7\}, 2\rangle$ and $\text{SoR}\langle\{r_2, r_3, r_7\}, 2\rangle$. The set of MER constraints corresponding to the SoR constraints is: $\text{MER}\langle\{r_1, r_3, r_7\}, 3\rangle$ and $\text{MER}\langle\{r_2, r_3, r_7\}, 3\rangle$. From Table IV, it can be verified that no user can access three or more rules in $\{r_1, r_3, r_7\}$ or $\{r_2, r_3, r_7\}$. Hence, the above MER constraints are satisfied, namely, $\mathcal{S}_1$ is satisfied.

2) Then, for $\mathcal{S}_2 = \langle\{p_1, p_2, p_3\}, 3\rangle$, the set of SoR constraints corresponding to $\mathcal{S}_2$ is: $\text{SoR}\langle\{r_1, r_3, r_4\}, 3\rangle$, $\text{SoR}\langle\{r_1, r_3, r_5\}, 3\rangle$, $\text{SoR}\langle\{r_1, r_3, r_6\}, 3\rangle$, $\text{SoR}\langle\{r_2, r_3, r_4\}, 3\rangle$, $\text{SoR}\langle\{r_2, r_3, r_5\}, 3\rangle$, and $\text{SoR}\langle\{r_2, r_3, r_6\}, 3\rangle$. The set of MER constraints obtained based on the SoR constraints is: $\text{MER}\langle\{r_1, r_3, r_4\}, 2\rangle$, $\text{MER}\langle\{r_1, r_3, r_5\}, 2\rangle$, $\text{MER}\langle\{r_1, r_3, r_6\}, 2\rangle$, $\text{MER}\langle\{r_2, r_3, r_4\}, 2\rangle$, $\text{MER}\langle\{r_2, r_3, r_5\}, 2\rangle$, and $\text{MER}\langle\{r_2, r_3, r_6\}, 2\rangle$. From Table IV, it can be seen that there exist users u_1 and u_3 who can access two rules in $\{r_1, r_3, r_4\}$, $\{r_1, r_3, r_5\}$, $\{r_2, r_3, r_4\}$, or $\{r_2, r_3, r_5\}$. Thus, the above MER constraints $\text{MER}\langle\{r_1, r_3, r_4\}, 2\rangle$, $\text{MER}\langle\{r_1, r_3, r_5\}, 2\rangle$, $\text{MER}\langle\{r_2, r_3, r_4\}, 2\rangle$, and $\text{MER}\langle\{r_2, r_3, r_5\}, 2\rangle$ are not satisfied. Hence, $\mathcal{S}_2$ is violated.

3) Finally, given $\mathcal{S}_3 = \langle \{p_1, p_2, p_3, p_4\}, 3 \rangle$, its corresponding SoR constraints are: $\text{SoR}(\{r_2, r_3, r_8\}, 3)$, $\text{SoR}(\{r_1, r_3, r_8\}, 3)$, and $\text{SoR}(\{r_1, r_2, r_3, r_8\}, 3)$. The set of MER constraints corresponding to the SoR constraints is: $\text{MER}(\{r_2, r_3, r_8\}, 2)$, $\text{MER}(\{r_1, r_3, r_8\}, 2)$, and $\text{MER}(\{r_1, r_2\}, 2)$, $\text{MER}(\{r_1, r_3\}, 2)$, $\text{MER}(\{r_1, r_8\}, 2)$, $\text{MER}(\{r_2, r_3\}, 2)$, $\text{MER}(\{r_2, r_8\}, 2)$ as well as $\text{MER}(\{r_3, r_8\}, 2)$. From Table IV, it can be observed that both users u_1 and u_3 can access two rules in $\{r_1, r_2\}$. Hence, the above MER constraint $\text{MER}(\{r_1, r_2\}, 2)$ is violated, implying the violation of $\mathcal{S}_3$, which is incidentally a false positive.

As we discussed in Example 2, $\mathcal{S}_3$ is satisfied rather than violated based on a brute-force method (i.e., DSoD-based method). However, in Example 3 a completely opposite checking result is obtained for $\mathcal{S}_3$ by using MER-based method. This shows that MER-based method is inadequate to check all SoD constraints. In fact, MER-based method can only be applied to the case that the cardinality $\diamond_i$ of an SoD constraint $\mathcal{S}_i$ is equal to 2 or $|\mathcal{P}_i|$ (see, e.g., $\mathcal{S}_1$ and $\mathcal{S}_2$ in Example 3), where $|\mathcal{P}_i|$ is the total number of permissions in the SoD constraint $\mathcal{S}_i$ and $i \in \mathbb{Z}_{|\mathcal{S}|}$.

Indeed, when using MER constraints to enforce SoD policies, one needs to determine whether a set of MER constraints are adequate to enforce an SoD constraint. The answer to this question depends on what permissions each user can complete, which, in turn, is determined by both the valid rule set of user (i.e., the set of rules that allows the user to perform operations on objects) and relevant rule set of permission in an SoD constraint (i.e., the set of rules that allows access to the permission in the SoD constraint). However, the implementation of MER-based method relies on only the relevant rule sets of permissions when transforming from SoD to MER through SoRs, while the valid rule sets of users are not considered. The lack of consideration of valid rule sets of users in the constraint conversion process ($\text{SoD} \rightarrow \text{SoR} \rightarrow \text{MER}$) may lead to the fact that the original rules which are not MERs are incorrectly classified as MERs. Take constraint $\mathcal{S}_3$ as an example. When both the valid rule sets of users and the relevant rule sets of permissions are taken into account (which is implemented by our approach in the sequel), a user is permitted to access non-MERs $\{r_1, r_2\}$ (since in this case, $\mathcal{S}_3$ is satisfied), while any user cannot access MERs $\{r_1, r_3\}$, $\{r_1, r_8\}$, $\{r_2, r_3\}$, $\{r_2, r_8\}$, or $\{r_3, r_8\}$. However, if only considering the relevant rule sets of permissions (which is carried out by MER-based method), then it is required that no user is allowed to access any two rules of r_1, r_2, r_3 , and r_8 (as specified in the set of MER constraints for $\mathcal{S}_3$ in case 3). Clearly, non-MERs $\{r_1, r_2\}$ are incorrectly classified as MERs in accordance with MER-based method.

In addition to the limited application range of MER-based method, it is practically infeasible since the generation of a set of SoR constraints for an SoD constraint is $\#\mathbf{P}$ -complete. Precisely speaking, MER-based method needs to exactly find every rule tuple in ABAC that corresponds to the permission tuple of each SoD constraint. It requires to conduct the Cartesian product of all relevant rule sets of the permissions in each SoD constraint. This is a brute-force method, leading to a combinatorial explosion problem.

V. AN EFFICIENT VERIFICATION APPROACH TO SOD CONSTRAINTS IN ABAC

In this section, we present an efficient method to check the satisfiability of SoD constraints in ABAC.

A. The Key to Check SoD Constraints

First, we explain the key to verify SoD constraints. Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. If this constraint is not satisfied, then it can be obtained that the permissions in $\mathcal{S}_i$ can be distributed among at least $\diamond \in \mathbb{Z}^+$ distinct users, where $\diamond < \diamond_i$. From the perspective of $\mathcal{S}_i$, at least $\diamond_i$ distinct users are expected to access all permissions in $\mathcal{S}_i$ in order to distribute the set of rules. However, from the viewpoint of ABAC, at most $\diamond$ different users are available to access all permissions in $\mathcal{S}_i$. This in turn implies that there must exist some users that can access different permissions involved in the same SoD constraint $\mathcal{S}_i$, leading to the fact that the minimal number of users together to finish all permissions in $\mathcal{S}_i$ is less than the cardinality $\diamond_i$. Hence, the key to check the satisfiability of SoD constraints is to identify which permissions can be completed by such users and which ones are not.

This can be done through the following steps: compute the related set of users assigned to each permission based on the relevant rule sets of permissions and valid rule sets of users at first, and then compare the related user sets of all permissions in the given SoD constraint. To the end, we can obtain which permissions can only be accessed by the users in a particular user set and which permissions can be visited by the users that belong to different user sets of distinct permissions. The related user set for each permission in an SoD constraint is formally defined as follows.

Definition 10: Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. Given a permission $p_k \in \mathcal{P}_i$, where $k \in \mathbb{Z}_{|\mathcal{P}_i|}$, its related user set is formally defined as

$$\Phi(p_k) = \{u \in \mathcal{U} \mid \Theta(u) \cap \Psi(p_k) \neq \emptyset\}$$

such that each user u in $\Phi(p_k)$ can complete p_k by accessing a rule belonging to both $\Theta(u)$ and $\Psi(p_k)$.

If $\forall s \in \mathbb{Z}_{|\mathcal{P}_i|}$ satisfies $s \neq k$ and $\Phi(p_s) \neq \Phi(p_k)$, it holds that $\Phi(p_s) \cap \Phi(p_k) = \emptyset$, then permission p_k can only be accessed by the users in $\Phi(p_k)$. We denote permission p_k as a non-conflict-user permission if it satisfies the above conditions. Conversely, a conflict-user permission in $\mathcal{P}_i$ can be finished by the users belonging to different user sets of distinct permissions involved in the same SoD constraint as p_k if it does not satisfy the above conditions. The formal definitions of non-conflict-user and conflict-user permissions are stated as follows.

Definition 11: Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. Permission p_k with $k \in \mathbb{Z}_{|\mathcal{P}_i|}$ is denoted as a non-conflict-user (resp., conflict-user) permission with respect to $\mathcal{S}_i$ if the following conditions hold (resp., do not hold): $\forall s \in \mathbb{Z}_{|\mathcal{P}_i|}$ satisfying $s \neq k$ and $\Phi(p_s) \neq \Phi(p_k)$, it holds that $\Phi(p_s) \cap \Phi(p_k) = \emptyset$. We use $\text{NC}(\mathcal{P}_i)$ and $\text{C}(\mathcal{P}_i)$ to represent the sets of non-conflict-user permissions and conflict-user permissions of $\mathcal{S}_i$, respectively. Clearly, $\forall p \in \text{NC}(\mathcal{P}_i)$ and $\forall p'$

$\in C(\mathcal{P}_i)$, it holds that $\Phi(p) \cap \Phi(p') = \emptyset$. Furthermore, $NC(\mathcal{P}_i) \cap C(\mathcal{P}_i) = \emptyset$ and $NC(\mathcal{P}_i) \cup C(\mathcal{P}_i) = \mathcal{P}_i$.

The following proposition shows that we can obtain the sets of non-conflict-user permissions and conflict-user permissions of SoD constraint $\mathcal{S}_i$ in an easy way.

Proposition 1: Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \Diamond_i \rangle$, where $i \in \mathbb{Z}_{|S|}$ and $\Diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. The computation of $NC(\mathcal{P}_i)$ and $C(\mathcal{P}_i)$ is in **P**.

Proof: The computation of $NC(\mathcal{P}_i)$ and $C(\mathcal{P}_i)$ needs to calculate $\Phi(p_k)$ for each $p_k \in \mathcal{P}_i$ and compare $\Phi(p_k)$ with $\Phi(p_s)$, where $s \in \mathbb{Z}_{|\mathcal{P}_i|}$ and $s \neq k$.

- The calculation of $\Phi(p_k)$ for each $p_k \in \mathcal{P}_i$ requires to compute $\Theta(u)$ and $\Psi(p_k)$ at first. This can be done in $O(|U||\mathcal{R}|)$ and $O(|\mathcal{O}||\mathcal{R}|)$ time units, respectively. Thus, the calculation of $\Phi(p_k)$ takes $O(|U||\mathcal{R}| + |\mathcal{O}||\mathcal{R}| + |\Psi(p_k)||U|)$ time units. For all $p_k \in \mathcal{P}_i$, the calculation of all $\Phi(p_k)$ together takes $O(|\mathcal{P}_i|(|U||\mathcal{R}| + |\mathcal{O}||\mathcal{R}|) + \sum_{k \in \mathbb{Z}_{|\mathcal{P}_i|}} |\Psi(p_k)||U|)$ time units.
- The comparison of $\Phi(p_k)$ with $\Phi(p_s)$, where $s \in \mathbb{Z}_{|\mathcal{P}_i|}$ and $s \neq k$, takes $O(|\mathcal{P}_i|^2)$ time units.

Thus, the total time complexity for computing $NC(\mathcal{P}_i)$ and $C(\mathcal{P}_i)$ is $O(|\mathcal{P}_i|(|U||\mathcal{R}| + |\mathcal{O}||\mathcal{R}|) + \sum_{k \in \mathbb{Z}_{|\mathcal{P}_i|}} |\Psi(p_k)||U| + |\mathcal{P}_i|^2)$, which is polynomial. ■

Next, we show how to compute the minimal number of users that can finish $NC(\mathcal{P}_i)$ and $C(\mathcal{P}_i)$, respectively. Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \Diamond_i \rangle$, where $i \in \mathbb{Z}_{|S|}$ and $\Diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. Since the intersection of related user sets of any two non-conflict-user and conflict-user permissions is empty, it can be obtained that calculating the minimal number of users together to complete all non-conflict-user permissions in $NC(\mathcal{P}_i)$ does not involve any user belonging to the related user sets of conflict-user permissions in $C(\mathcal{P}_i)$, and vice versa. In other words, the minimal numbers of users together to complete all permissions in $NC(\mathcal{P}_i)$ and $C(\mathcal{P}_i)$ can be computed separately. We use Δ_i and ∇_i to represent the minimal numbers of users to complete all permissions in $NC(\mathcal{P}_i)$ and $C(\mathcal{P}_i)$, respectively. Now, we show how to obtain Δ_i and ∇_i individually.

1) Computation of Δ_i .

Algorithm 1 shows the detailed procedures to compute Δ_i . It follows the idea of comparing the related user sets of each pair of non-conflict-user permissions in $NC(\mathcal{P}_i)$ so as to obtain the total numbers of different related user sets. Since different permissions in $NC(\mathcal{P}_i)$ may have the same allowed users, the minimal number of users required to complete all permissions in $NC(\mathcal{P}_i)$ usually is not greater than $|NC(\mathcal{P}_i)|$.

Proposition 2: Algorithm 1 is in **P**.

Proof: Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \Diamond_i \rangle$, where $i \in \mathbb{Z}_{|S|}$ and $\Diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. The computation of Δ_i needs to obtain $NC(\mathcal{P}_i)$ at first. This can be done in polynomial time by Proposition 1. Then, computing Δ_i requires to compare each pair of permissions in $NC(\mathcal{P}_i)$, which can be done in $O(|NC(\mathcal{P}_i)|^2)$ time units. Hence, the whole process is of polynomial complexity. ■

2) Computation of ∇_i .

Let the set of indexes for all permissions in $C(\mathcal{P}_i)$ be Γ_i . Formally

$$\Gamma_i = \{k \in \mathbb{Z}_{|\mathcal{P}_i|} \mid p_k \in C(\mathcal{P}_i)\}.$$

Algorithm 1: Computing Δ_i .

```

1 Input: An ABAC model and an SoD constraint  $\mathcal{S}_i = \langle \mathcal{P}_i, \Diamond_i \rangle$ , where  $i \in \mathbb{Z}_{|S|}$  and  $\Diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$ ;
2 Output:  $\Delta_i$ ;
3 Initialization:  $\Delta_i = 0$ ;
4 Compute  $NC(\mathcal{P}_i)$ ;
5 while  $NC(\mathcal{P}_i) \neq \emptyset$  do
6   Select the first permission  $p_k$  from  $NC(\mathcal{P}_i)$ ,
   where  $k \in \mathbb{Z}_{|\mathcal{P}_i|}$ ;
7   if  $\nexists p_s \in NC(\mathcal{P}_i)$ , where  $s \in \mathbb{Z}_{|\mathcal{P}_i|}$  and  $s \neq k$ ,
   such that  $\Phi(p_s) = \Phi(p_k)$  then
8      $\Delta_i = \Delta_i + 1$ ;
9    $NC(\mathcal{P}_i) = NC(\mathcal{P}_i) \setminus \{p_k\}$ ;
10 Return  $\Delta_i$ .
```

Let the set of users that are related to all permissions in $C(\mathcal{P}_i)$ be Π_i and the set of indexes for all users in $\Pi(C(\mathcal{P}_i))$ be Λ_i . Formally,

$$\Pi_i = \bigcup_{k \in \Gamma_i} \Phi(p_k).$$

$$\Lambda_i = \{j \in \mathbb{Z}_{|U|} \mid u_j \in \Pi_i\}.$$

Then, we have $\nabla_i = \sum_{j \in \Lambda_i} x_j$, where each x_j can be obtained based on the solution of the following 0–1 IP problem.

$$\min \sum_{j \in \Lambda_i} x_j \quad (1)$$

s.t.

$$\sum_{j \in \Lambda_i} \alpha_{j,k} x_j \geq 1, \forall k \in \Gamma_i \quad (2)$$

$$x_j \in \{0, 1\}, \forall j \in \Lambda_i \quad (3)$$

Here, we use coefficient $\alpha_{j,k} \in \{0, 1\}$ to represent whether there exists an access relation between user u_j and permission p_k in the ABAC system, where $j \in \Lambda_i$, and $k \in \Gamma_i$. Specifically, $\alpha_{j,k}$ takes value 1 if $u_j \in \Phi(p_k)$, i.e., user u_j can finish permission p_k by accessing certain rules; and 0, otherwise. And boolean variable $x_j \in \{0, 1\}$ denotes whether user u_j should be picked necessarily to complete one or more permissions in $C(\mathcal{P}_i)$ when computing the minimal number of users required to complete all conflict-user permissions in $C(\mathcal{P}_i)$. Specifically, x_j takes value 1 if u_j is required to complete at least one permission in $C(\mathcal{P}_i)$; and 0, otherwise.

Note that, solving 0–1 IP problem is **NP**-hard in theory. This implies that using 0–1 IP (1)–(3) to compute ∇_i for SoD constraint $\mathcal{S}_i$ is of exponential complexity in the worst case. Nevertheless, since 0–1 IP is a standard optimization tool, we can use the off-the-shelf commercial software suites (such as CPLEX [13]) to solve it efficiently. Moreover, since the permissions in an SoD constraint are partitioned into non-conflict-user and conflict-user ones, the number of conflict-user permissions used in IP (1)–(3) is much smaller than the total number of permissions in the SoD constraint. As a result, the whole search space of IP (1)–(3) is greatly reduced when computing ∇_i .

B. The Procedures to Check SoD Constraints

Now, we show how to determine the satisfiability of an SoD constraint $\mathcal{S}_i$ based on Δ_i and ∇_i .

If $C(\mathcal{P}_i) = \emptyset$, then $NC(\mathcal{P}_i) = \mathcal{P}_i$. That is, each permission in an SoD constraint can only be finished by the users belonging to some particular user sets. Under this case, the satisfiability of $\mathcal{S}_i$ can be easily determined based on Δ_i .

Proposition 3: Consider SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$ satisfying $C(\mathcal{P}_i) = \emptyset$. $\mathcal{S}_i$ is satisfied if $\Delta_i \geq \diamond_i$; and violated, otherwise.

Proof: Due to $C(\mathcal{P}_i) = \emptyset$, it holds that $\mathcal{P}_i = NC(\mathcal{P}_i)$. Since the minimal number of users that together accomplish all permissions in $NC(\mathcal{P}_i)$ is Δ_i , we can obtain that at least Δ_i users are required together to perform all permissions in $\mathcal{P}_i$. Because $\Delta_i \geq \diamond_i$ (resp., $\Delta_i < \diamond_i$), it can be concluded that $\mathcal{S}_i$ is satisfied (resp., violated). ■

Proposition 3 shows that for an SoD constraint $\mathcal{S}_i$ whose conflict-user permission set $C(\mathcal{P}_i)$ is empty (we denote such constraint as $\emptyset$ -SoD constraint), its satisfiability can be directly checked by comparing $\diamond_i$ with Δ_i . This procedure can be done in polynomial time by Proposition 2.

Then, we show that under certain conditions, the SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$ satisfying $C(\mathcal{P}_i) \neq \emptyset$ can also be verified in polynomial time.

Let the number of different permissions that a user $u \in \Pi_i$ can access in $C(\mathcal{P}_i)$ be $\xi(u)$. Formally,

$$\xi(u) = |\{p \in C(\mathcal{P}_i) \mid \Theta(u) \cap \Psi(p) \neq \emptyset\}|.$$

By comparing $\xi(u)$ of all users in Π_i , we can obtain the maximal number of permissions, i.e., upper bound, a certain user can complete among all users:

$$\Xi_i = \max_{j \in \Lambda_i} (\xi(u_j)).$$

Proposition 4: Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$ satisfying $C(\mathcal{P}_i) \neq \emptyset$. If $|C(\mathcal{P}_i)| + 1 - \Xi_i < \diamond_i - \Delta_i$, then $\mathcal{S}_i$ is violated.

Proof: Because the minimal number of users that can accomplish all permissions in $NC(\mathcal{P}_i)$ is Δ_i , it can be obtained that at least $\diamond_i - \Delta_i$ users are needed to complete all permissions in $C(\mathcal{P}_i)$. Based on the definition of Ξ_i , it can be derived that there exists a user who can complete Ξ_i conflict-user permissions. Hence, at most $|C(\mathcal{P}_i)| - \Xi_i$ users are required to finish the rest conflict-user ones. As a result, the maximal number of users together to perform all $|C(\mathcal{P}_i)|$ conflict-user permissions is $1 + |C(\mathcal{P}_i)| - \Xi_i$. This is less than $\diamond_i - \Delta_i$. Hence, $\mathcal{S}_i$ is violated. ■

For simplicity, we denote an SoD constraint satisfying condition $|C(\mathcal{P}_i)| + 1 - \Xi_i + \Delta_i < \diamond_i$ as $\blacktriangle$ -SoD constraint.

Theorem 2: SC-SoD with respect to $\emptyset$ -SoD or $\blacktriangle$ -SoD constraints is in **P**.

Proof: For each SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, we first determine whether $C(\mathcal{P}_i) = \emptyset$. If so, then $\mathcal{S}_i$ is determined as a $\emptyset$ -SoD constraint so that its satisfiability can be checked in polynomial time by Propositions 1 and 2. Otherwise, we compute $\xi(u)$ for each user $u \in \Pi_i$ so as to obtain Ξ_i at first, and then determine whether condition $|C(\mathcal{P}_i)| + 1 - \Xi_i < \diamond_i - \Delta_i$

Algorithm 2: Solving SC-SoD.

```

1 Input: An ABAC model and SoD constraint set  $\mathcal{S}$ ;
2 Output: Checking results and SoD violation set  $\mathcal{VS}$ ;
3 Initialization:  $\mathcal{VS} = \emptyset$ ;
4 while  $\mathcal{S} \neq \emptyset$  do
5   Select the first constraint  $\mathcal{S}_i$  from  $\mathcal{S}$ , where  $i \in \mathbb{Z}_{|\mathcal{S}|}$ ;
6   Compute  $NC(\mathcal{P}_i)$ ,  $C(\mathcal{P}_i)$ , and  $\Delta_i$ ;
7   if  $C(\mathcal{P}_i) = \emptyset$  then
8     if  $\Delta_i \geq \diamond_i$  then
9       Return  $\mathcal{S}_i$  is satisfied;
10    else
11      Return  $\mathcal{S}_i$  is violated;
12       $\mathcal{VS} = \mathcal{VS} \cup \{\mathcal{S}_i\}$ ;
13  else
14    Calculate  $\Xi_i$ ;
15    if  $|C(\mathcal{P}_i)| + 1 - \Xi_i < \diamond_i - \Delta_i$  then
16      Return  $\mathcal{S}_i$  is violated;
17       $\mathcal{VS} = \mathcal{VS} \cup \{\mathcal{S}_i\}$ ;
18    else
19      Obtain  $\nabla_i$  by 0-1 IP (1) - (3);
20      if  $\Delta_i + \nabla_i \geq \diamond_i$  then
21        Return  $\mathcal{S}_i$  is satisfied;
22      else
23        Return  $\mathcal{S}_i$  is violated;
24         $\mathcal{VS} = \mathcal{VS} \cup \{\mathcal{S}_i\}$ ;
25   $\mathcal{S} = \mathcal{S} \setminus \{\mathcal{S}_i\}$ ;
26 Return  $\mathcal{VS}$ .

```

holds. This process has a time complexity of $O(|C(\mathcal{P}_i)| |\Pi(i)|)$, which is polynomial. ■

By Theorem 2, SC-SoD with respect to $\emptyset$ -SoD or $\blacktriangle$ -SoD constraints can be solved easily. This implies that the proposed approach is particularly well suited for the SoD constraints whose permissions have little overlapping users with each other.

Finally, we show how to check an SoD constraint that is neither $\emptyset$ -SoD nor $\blacktriangle$ -SoD constraint.

Proposition 5: Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$ that is neither $\emptyset$ -SoD nor $\blacktriangle$ -SoD constraint. $\mathcal{S}_i$ is satisfied (resp., violated) if and only if $\Delta_i + \nabla_i \geq \diamond_i$ (resp., $\Delta_i + \nabla_i < \diamond_i$).

Proof: Based on the definitions of Δ_i and ∇_i , $\Delta_i + \nabla_i$ exactly means the total number of users to complete all the permissions, i.e., $\mathcal{P}_i$, in $\mathcal{S}_i$. Hence, this statement holds obviously. ■

Based on the above discussions, we present the detailed procedures to check the satisfiability of all SoD constraints in Algorithm 2.

Example 4: Let us consider again the ABAC model in Example 1 and the three SoD constraints $\mathcal{S}_1$, $\mathcal{S}_2$, and $\mathcal{S}_3$. We illustrate how to check the satisfaction of $\mathcal{S}_1$, $\mathcal{S}_2$, and $\mathcal{S}_3$ by using Algorithm 2.

- 1) Consider $\mathcal{S}_1 = \langle \{p_1, p_2, p_3\}, 2 \rangle$ at first. The related user set for each permission in $\mathcal{S}_1$ is $\Phi(p_1) = \{u_2\}$, $\Phi(p_2) = \{u_2\}$, and $\Phi(p_3) = \{u_1, u_3\}$. Hence, $NC(\mathcal{P}_1) = \{p_1, p_2,$

$p_3\}$ and $C(\mathcal{P}_1) = \emptyset$. Based on Algorithm 1, we can obtain $\Delta_1 = 2$. Then, based on the fact that $C(\mathcal{P}_1) = \emptyset$ and $\Delta_1 = \Diamond_1 = 2$, $\mathcal{S}_1$ is determined as satisfied as a $\emptyset$ -SoD.

- 2) For the second constraint $\mathcal{S}_2 = \langle \{p_1, p_2, p_3\}, 3 \rangle$, we first obtain the related user set for each permission in $\mathcal{S}_2$ as follows: $\Phi(p_1) = \{u_1, u_3, u_4\}$, $\Phi(p_2) = \{u_2\}$, and $\Phi(p_3) = \{u_1, u_3\}$. Accordingly, $NC(\mathcal{P}_2) = \{p_2\}$ and $C(\mathcal{P}_2) = \{p_1, p_3\}$. By using Algorithm 1, it can be obtained that $\Delta_2 = 1$. Then, we have $\Xi_2 = \max(\xi(u_1) = |\{p_1, p_3\}|, \xi(u_3) = |\{p_1, p_3\}|, \xi(u_4) = |\{p_1\}|) = 2$. Finally, Algorithm 2 returns the result that $\mathcal{S}_2$ is violated as a $\blacktriangle$ -SoD due to $|C(\mathcal{P}_2)| + 1 - \Xi_2 = 2 + 1 - 2 = 1 < \Diamond_2 - \Delta_2 = 3 - 1 = 2$. Correspondingly, $\mathcal{VS} = \{\mathcal{S}_2\}$.
- 3) Third, consider the last constraint $\mathcal{S}_3 = \langle \{p_1, p_2, p_3, p_4\}, 3 \rangle$. The related user set for each permission in $\mathcal{S}_3$ is $\Phi(p_1) = \{u_1, u_3\}$, $\Phi(p_2) = \{u_5\}$, $\Phi(p_3) = \{u_1, u_3, u_5\}$, and $\Phi(p_4) = \{u_2\}$. Thus, $NC(\mathcal{P}_3) = \{p_4\}$ and $C(\mathcal{P}_3) = \{p_1, p_2, p_3\}$. Based on Algorithm 1, we can obtain $\Delta_3 = 1$. Next, we have $\Xi_3 = \max(\xi(u_1) = |\{p_1, p_3\}|, \xi(u_3) = |\{p_1, p_3\}|, \xi(u_5) = |\{p_2, p_3\}|) = 2$. Because $C(\mathcal{P}_3) \neq \emptyset$ and $|C(\mathcal{P}_3)| + 1 - \Xi_3 = \Diamond_3 - \Delta_3 = 2$, $\mathcal{S}_3$ is neither a $\emptyset$ -SoD nor a $\blacktriangle$ -SoD such that either Proposition 3 or 4 is no longer applicable. To resort to Proposition 5, we need to further compute ∇_3 to check the satisfiability of $\mathcal{S}_3$. Based on 0-1 IP (1)-(3), it can be obtained that $\nabla_3 = 2$. About 2 ms are taken to obtain ∇_3 by using CPLEX optimization package in the Java environment running on a laptop equipped with an Intel i7 CPU at 2.30 GHz and with 16 GB of RAM. Finally, since $\Delta_3 + \nabla_3 = \Diamond_3 = 3$, we can conclude that $\mathcal{S}_3$ is satisfied.

VI. SOLUTION TO SoD VIOLATIONS BASED ON IP

In this section, we show how to solve SoD violations so as to make all SoD constraints satisfied. This is of great significance to guarantee not only the satisfaction of authorization security but also the enforceability of ABAC.

If there exists an SoD violation, then we know that some users exist in the ABAC that can complete different conflict-user permissions that belong to the same SoD constraints. Hence, we can solve SoD violations in the way that certain users should not accomplish particular permissions belonging to the SoD violations. This can be achieved by preventing users from accessing certain rules that belong to not only the valid rule sets of the considered users but also the relevant rule sets of particular conflict-user permissions in SoD violations.

Consider an SoD constraint $\mathcal{S}_i = \langle \mathcal{P}_i, \Diamond_i \rangle$ that is violated, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\Diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. If $C(\mathcal{P}_i) = \emptyset$, then the SoD violation cannot be directly solved by removing certain rules that users can access. This is because for such SoD violation there exists no user in the ABAC that can perform any conflict-user permission since no one exists. The forbiddance of accesses of users to some rules may lead to the fact that the permissions related to the specified rules cannot be completed by any user. A possible solution to this problem is to reassign access among users, rules, and permissions in ABAC. Such issue is beyond the scope of this paper.

In this paper, we only consider solving the SoD violation whose conflict-user permission set is not empty. In the following, we present an approach to formulating the violation resolving problem into a 0-1 IP.

First, we define the variables used in the IP. Consider an SoD constraint $\mathcal{S}_i$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$. Given a permission $p_k \in \mathcal{P}_i$ and a user $u_j \in \Phi(p_k)$, where $j \in \mathbb{Z}_{|\mathcal{U}|}$ and $k \in \mathbb{Z}_{|\mathcal{P}_i|}$, we use $\Upsilon(j, k)$ to denote the set of rules, where each element in this set belongs to both $\Theta(u_j)$ and $\Psi(p_k)$. Formally

$$\Upsilon(j, k) = \Theta(u_j) \cap \Psi(p_k).$$

Let coefficient $\mu_{j,h} = 1$ if $\exists p_k \in \mathcal{P}_i$ such that $u_j \in \Phi(p_k)$ and $r_h \in \Upsilon(j, k)$, i.e., there exists a user u_j who can finish a permission p_k in SoD constraint $\mathcal{S}_i$ by accessing a rule r_h in rule set $\Upsilon(j, k)$. Otherwise, we set $\mu_{j,h} = 0$. Accordingly, let $y_{j,h} \in \{0, 1\}$ be a boolean variable to indicate whether user u_j can access rule r_h when solving SoD violations. Specifically, $y_{j,h}$ takes value 1 if user u_j is allowed to access rule r_h such that s/he can complete permission p_k by accessing rule r_h , where $u_j \in \Phi(p_k)$ and $r_h \in \Upsilon(j, k)$; and 0, otherwise.

Let the set of indexes for all SoD violations in SoD violation set $\mathcal{VS}$ be Δ . Formally

$$\Delta = \{i \in \mathbb{Z}_{|\mathcal{S}|} \mid \mathcal{S}_i \in \mathcal{VS}\}.$$

Then, given the above variables, we present the following 0-1 IP to solve all SoD violations.

1) The objective function is given at first such that SoD violations are solved by preventing users from accessing rules with the criterion that the accesses between users and rules are preserved as more as possible.

$$\max \sum_{j \in \mathbb{Z}_{|\mathcal{U}|}} \sum_{h \in \mathbb{Z}_{|\mathcal{R}|}} \mu_{j,h} y_{j,h} \quad (4)$$

s.t.

2) In order to ensure that each conflict-user permission in an SoD violation should be accessed by one user, we can generate the following constraints.

$$\sum_{j \in \Lambda_i} \max_{r_h \in \Upsilon(j, k)} (\mu_{j,h} y_{j,h}) = 1, \forall i \in \Delta, \forall k \in \Gamma_i \quad (5)$$

3) In order to make each SoD violation satisfied, the minimal number of users together to perform all conflict-user permissions in an SoD violation should be greater than or equal to the required cardinality constraint. Thus, we can obtain the following constraints.

$$\sum_{j \in \Lambda_i} z_j^i \geq \Diamond_i - \Delta_i, \forall i \in \Delta \quad (6)$$

$$z_j^i = \max_{k \in \Gamma_i, r_h \in \Upsilon(j, k)} (\mu_{j,h} y_{j,h}), \forall i \in \Delta, \forall j \in \Lambda_i \quad (7)$$

4) To guarantee that each permission in a satisfied SoD constraint should be completed by at least one user via accessing certain rules, we add the following constraints.

$$\sum_{j \in \mathbb{Z}_{|\mathcal{U}|}} \max_{r_h \in \Upsilon(j, k)} (\mu_{j,h} y_{j,h}) \geq 1, \forall i \in \mathbb{Z}_{|\mathcal{S}|} \setminus \Delta, \forall k \in \mathbb{Z}_{|\mathcal{P}_i|} \quad (8)$$

5) The value ranges of variables are given as follows.

$$y_{j,h} \in \{0, 1\}, \forall j \in \mathbb{Z}_{|\mathcal{U}|}, \forall h \in \mathbb{Z}_{|\mathcal{R}|} \quad (9)$$

We then show how to apply the solution of 0–1 IP (4)–(9) to the ABAC model. Consider $y_{j,h}$ in the obtained solution. If $y_{j,h} = 1$, then the access relation between user u_j and rule r_h remains unchanged. Hence, the valid rule set of u_j is not changed. Otherwise, user u_j cannot access r_h . Thus, the valid rule set of u_j is changed to $\Theta(u_j) = \Theta(u_j) \setminus \{r_h\}$.

Proposition 6: Consider a set of SoD violations $\mathcal{VS}$, each of which is in the form $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. If 0–1 IP (4)–(9) can admit a solution for $\mathcal{VS}$, then after applying this solution to the ABAC model, all SoD constraints are satisfied.

Proof: We first prove that all SoD violations can be corrected based on the solution of 0–1 IP (4)–(9). Consider an SoD violation $\mathcal{S}_i = \langle \mathcal{P}_i, \diamond_i \rangle$, where $i \in \mathbb{Z}_{|\mathcal{S}|}$ and $\diamond_i \in \mathbb{Z}_{|\mathcal{P}_i|}$. If 0–1 IP (4)–(9) admits a solution, then it can be obtained that each conflict-user permission in $C(\mathcal{P}_i)$ can be accessed by one user (due to Constraint (5)) and that the minimal number of users to complete all conflict-user permissions in $C(\mathcal{P}_i)$ is not less than $\diamond_i - \triangle_i$ (due to Constraints (6) and (7)). Since the minimal number of users that can together accomplish all non-conflict-user permissions in $NC(\mathcal{P}_i)$ is $\triangle_i$, we can derive that at least $\diamond_i - \triangle_i + \triangle_i = \diamond_i$ users together can complete all permissions in $C(\mathcal{P}_i) \cup NC(\mathcal{P}_i) = \mathcal{P}_i$. This implies that $\mathcal{S}_i$ is satisfied. The same deduction holds for each SoD violation. Hence, all SoD violations can be corrected based on the solution of 0–1 IP (4)–(9).

Then, we show that solving SoD violations based on 0–1 IP (4)–(9) does not affect the satisfiability of any other SoD constraints, i.e., after SoD violations are satisfied, all SoD constraints are satisfied. First, due to Constraints (6) and (8), it can be obtained that for any permission in any SoD constraint, there exists at least one user who can perform the permission by accessing certain rules. Second, since SoD violations are solved by forbidding users to access certain rules, no more access relation between any user and permission is added when solving SoD violations. This means that there does not exist the case that a satisfied SoD constraint becomes violated due to the fact that some users can access more different permissions involved in the same SoD constraint after applying the solution of 0–1 IP (4)–(9) on the ABAC model. Therefore, all SoD constraints are satisfied after all SoD violations are solved. ■

Example 5: Consider the SoD violation $\mathcal{S}_2 = \langle \{p_1, p_2, p_3\}, 3 \rangle$ in Example 4. Based 0–1 IP (4)–(9), we can obtain a solution such that $y_{1,4} = y_{1,5} = y_{3,1} = y_{3,2} = y_{4,6} = 0$, while the values of the other variable are 1. About 62 ms are spent to obtain this solution in the given CPLEX platform. Based on this solution, it can be obtained that:

- 1) user u_1 is forbidden to access rules r_4 and r_5 (due to $y_{1,4} = y_{1,5} = 0$), and thus s/he cannot complete permission p_1 of $\mathcal{S}_2$,
- 2) user u_3 is not allowed to access rules r_1 and r_2 (due to $y_{3,1} = y_{3,2} = 0$) such that s/he cannot finish permission p_3 of $\mathcal{S}_1$, permission p_3 of $\mathcal{S}_2$, and permissions p_1 and p_3 of $\mathcal{S}_3$,

- 3) user u_4 is denied to perform permission p_1 of $\mathcal{S}_2$ by preventing from accessing rule r_6 (due to $y_{4,6} = 0$), and the access relations between other users and rules remain unchanged.

Consequently, the valid rule sets of users u_1 , u_3 , and u_4 , i.e., $\Theta(u_1) = \{r_1, r_2, r_4, r_5\}$, $\Theta(u_3) = \{r_1, r_2, r_4, r_5\}$, and $\Theta(u_4) = \{r_6\}$, are changed to $\Theta(u_1) = \{r_1, r_2, r_4, r_5\} \setminus \{r_4, r_5\} = \{r_1, r_2\}$, $\Theta(u_3) = \{r_1, r_2, r_4, r_5\} \setminus \{r_1, r_2\} = \{r_4, r_5\}$, and $\Theta(u_4) = \{r_6\} \setminus \{r_6\} = \emptyset$, respectively. By doing so, all SoD constraints are satisfied.

VII. EXPERIMENTAL RESULTS

In this section, we first show the fact that the computational complexity of our approach (denoted as PSoD-based method since it relies on the partition of permissions in SoD constraints) to satisfaction checking of $\emptyset$ -SoD or $\blacktriangle$ -SoD constraints increases linearly with respect to the numbers of users, rules, SoD constraints, permissions in SoD constraints, and rules in the permissions of SoD constraints, respectively. To simplify notions, we use userNum, ruleNum, SoDNum, PermissionNum, and RuleNum to represent the numbers of users, rules, SoD constraints, permissions in SoD constraints, and rules in the permissions of SoD constraints, respectively. The experimental settings are detailed as follows.

- We develop a Java software to apply the PSoD-based approach presented in Section IV on a laptop with Intel i7 CPU at 2.30 GHz and with 16 GB of RAM.
- The data set used for the analysis is explained as follows: each randomly constructed ABAC model is equipped with 10 objects, each of which has 5 well-defined attributes, and each user is associated with 10 user attributes; userNum, ruleNum, SoDNum, PermissionNum, and RuleNum are generated randomly based on the implementation. Each SoD constraint is either $\emptyset$ -SoD or $\blacktriangle$ -SoD constraint. Each result is reported as an average over 50 runs.

1) Evaluate the effect of the increase in userNum on the execution time required for checking SoD constraints based on PSoD-based method.

Fig. 1 presents the impact of the increase in the number of users from 10 to 100 on the execution time required for checking SoD constraints when setting ruleNum = 50, SoDNum = 30, PermissionNum = 30 in each SoD constraint, and RuleNum = 10 for each permission in each SoD constraint. It can be seen from Fig. 1 that by using PSoD-based method, the execution time required to check SoD constraints increases linearly with respect to the numbers of users.

2) Evaluate the effect of the increase in ruleNum on the execution time required for checking SoD constraints based on PSoD-based method.

Fig. 2 shows that with the variation of ruleNum from 10 to 100 and the invariance of userNum = 100, SoDNum = 30, PermissionNum = 30 in each SoD constraint, and RuleNum = 10 for each permission in each SoD constraint, the time complexity of PSoD-based method for checking SoD constraints grows linearly with the increase of the number of rules.

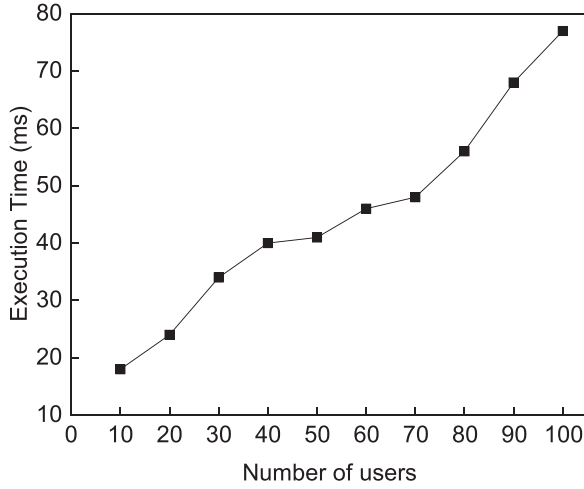


Fig. 1. Evaluate the effect of the increase in userNum on the execution time required for checking SoD constraints based on PSoD-based method.

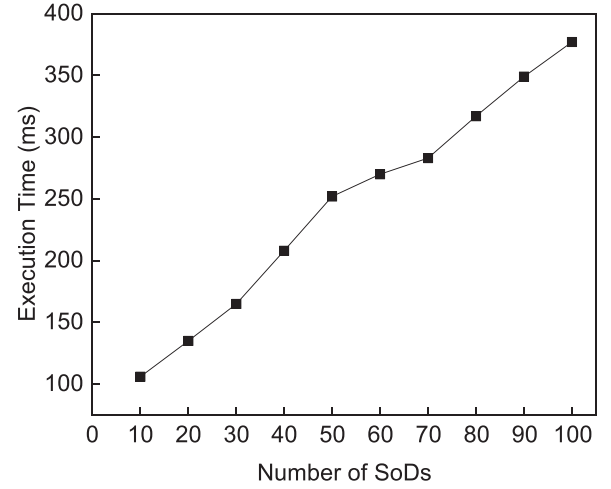


Fig. 3. Evaluate the effect of the increase in SoDNum on the execution time required for checking SoD constraints based on PSoD-based method.

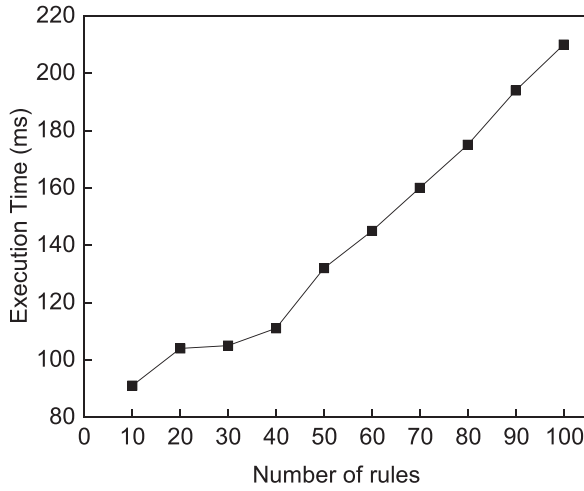


Fig. 2. Evaluate the effect of the increase in ruleNum on the execution time required for checking SoD constraints based on PSoD-based method.

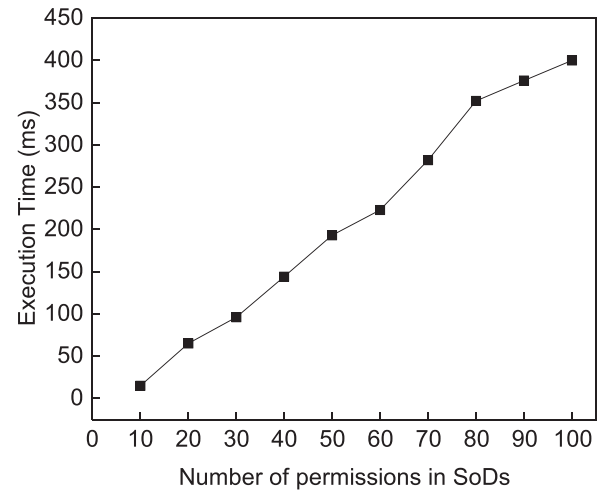


Fig. 4. Evaluate the effect of the increase in PermissionNum in the SoD constraints on the execution time required for checking SoD constraints based on PSoD-based method.

3) Evaluate the effect of the increase in SoDNum on the execution time required for checking SoD constraints based on PSoD-based method.

In this experiment, we set userNum = 100, ruleNum = 100, PermissionNum = 30 in each SoD constraint, and RuleNum = 10 for each permission in each SoD constraint, while SoDNum varies from 10 to 100.

It can be observed from Fig. 3, with the linear increase of the number of SoD constraints, the time spent to check the satisfaction of SoD constraints through PSoD-based method also increases linearly.

4) Evaluate the effect of the increase in PermissionNum on the execution time required for checking SoD constraints based on PSoD-based method.

The experimental cases used here are with the following settings: userNum = 50, ruleNum = 50, SoDNum = 20, and RuleNum = 10 for each permission in each SoD constraint, while PermissionNum = 10, 20, ..., 100. Fig. 4 shows the

experimental results, which demonstrate that the computational complexity of PSoD-based method for checking SoD constraints grows linearly with respect to the increase in the number of permissions in SoD constraints.

5) Evaluate the effect of the increase in RuleNum on the execution time required for checking SoD constraints based on PSoD-based method.

Fig. 5 evaluates the effect of the increase in RuleNum on the execution time required for checking SoD constraints by PSoD-based method. Specifically, by setting userNum = 100, ruleNum = 100, SoDNum = 20, and PermissionNum = 30 in each SoD constraint and varying RuleNum for each permission in each SoD constraint from 10 to 100, it can be seen from Fig. 5 that the execution time required to check SoD constraints increases linearly with respect to RuleNum.

Based on the experimental results in Figs. 1–5, it can be concluded that with the linear increase in userNum, ruleNum,

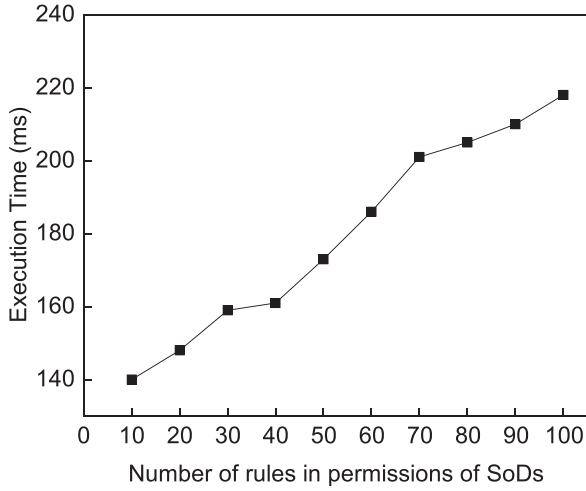


Fig. 5. Evaluate the effect of the increase in RuleNum in the permissions of SoD constraints on the execution time required for checking SoD constraints based on PSoD-based method.

SoDNum, PermissionNum, and RuleNum, the time required to solve the satisfaction checking problem of $\emptyset$ -SoD and $\blacktriangle$ -SoD constraints based on PSoD-based method grows linearly.

Then, we evaluate the effect of the variations in userNum, ruleNum, SoDNum, PermissionNum, and RuleNum on the execution times required for checking SoD constraints by our PSoD-based method, MER-based method (proposed in [14]), and DSoD-based method (used in [17], [22], [36], [37]). MER-based and DSoD-based methods are two mainstream methods for SoD constraint verification that are widely used in the existing literature [14], [15], [17], [22], [23], [24], [36], [37]. Hence, we compare our PSoD-based method with these two typical methods. Here, we set the cardinality of each SoD constraint as 2 or the number of permissions in the SoD constraint since MER-based method can only handle such SoD constraints. The experiments carried out to show the complexities of the three compared approaches are run on the same platform given above.

1) Evaluate the effect of the increase in userNum, ruleNum, and SoDNum on the execution time required for checking SoD constraints by PSoD-based, MER-based, and DSoD-based methods.

In this experiment, for each case we set PermissionNum = 3 in each SoD constraint and RuleNum = 3 for each permission in each SoD constraint. The values of userNum, ruleNum, and SoDNum in the cases are set as follows:

- userNum = ruleNum = 10, and SoDNum = 5;
- userNum = ruleNum = 20, and SoDNum = 15;
- userNum = ruleNum = 30, and SoDNum = 20;
- userNum = ruleNum = 40, and SoDNum = 25;
- userNum = ruleNum = 50, and SoDNum = 30.

From Fig. 6, it can be seen that with the linear increase of userNum, ruleNum, and SoDNum, the time required to check SoD constraints based on PSoD-based, MER-based, or DSoD-based method grows linearly.

2) Evaluate the effect of the increase in PermissionNum and RuleNum on the execution time required for checking SoD

constraints by PSoD-based, MER-based, and DSoD-based methods.

The test cases are with the following settings: both PermissionNum and RuleNum range from 2 to 8, and userNum = ruleNum = 20, and SoDNum = 10.

Consider the complexities of the three methods, which are given as follows:

- The complexity of PSoD-based method is $O(\text{SoDNum} \cdot (\text{PermissionNum}_1 \cdot \text{RuleNum}_1 \cdot \text{userNum} + \text{PermissionNum}_2 \cdot \text{RuleNum}_2 \cdot 2^{\text{RuleNum}_2}))$, which can be simplified to $O(\text{SoDNum} \cdot \text{PermissionNum}_2 \cdot \text{RuleNum}_2 \cdot 2^{\text{RuleNum}_2})$, where PermissionNum₁ and PermissionNum₂ represent the numbers of non-conflict-user and conflict-user permissions, respectively, such that PermissionNum₁ + PermissionNum₂ = PermissionNum; and RuleNum₁ and RuleNum₂ indicate the numbers of rules related to non-conflict-user and conflict-user permissions, respectively, such that RuleNum₁ + RuleNum₂ = RuleNum;
- The complexity of MER-based method is $O(\text{SoDNum} \cdot \text{RuleNum}^{\text{PermissionNum}})$; and
- The complexity of DSoD-based method is $O(\text{SoDNum} \cdot \text{PermissionNum} \cdot \text{RuleNum} \cdot 2^{\text{RuleNum}})$.

Although any of them increases exponentially with the increase of PermissionNum and RuleNum in the worst case, PSoD-based method performs better than the others since it can verify both $\emptyset$ -SoD and $\blacktriangle$ -SoD constraints in **P** (as proved in Theorem 2), leading to the fact that RuleNum₂ ≤ RuleNum as well as RuleNum₂ · 2^{RuleNum₂} ≤ RuleNum^{PermissionNum} and RuleNum₂ · 2^{RuleNum₂} ≤ RuleNum · 2^{RuleNum}. The experimental results depicted in Fig. 7 also validate this fact. It can be easily seen that when using either MER-based or DSoD-based method to check the satisfaction of SoD constraints, the required time increases exponentially with the increase of PermissionNum and RuleNum, and both are larger than that of PSoD-based method. This trend arises due to the fact that either MER-based or DSoD-based method requires to compute the Cartesian product of all relevant rule sets of all permissions in every SoD constraint. Such procedure is a brute force method, leading to an ineluctable combinatorial explosion problem.

The above statements are further intuitively illustrated by the experimental results given in Table VIII. Here, for each case in Table VIII, we set the cardinality of each SoD constraint as 2 or the number of permissions in the SoD constraint since MER-based method can only handle such SoD constraints. Furthermore, we use PSoD time, MER time, and DSoD time to record the times (ms) spent by PSoD-based method, MER-based method, and DSoD-based method, respectively, to check the satisfaction of SoD constraints. In Table VIII (and the subsequent Tables IX and X), the columns with the least time are highlighted in bold for clarity.

As shown in Table VIII, when the values of PermissionNum and RuleNum are small (Cases 1, 2, 4, 5, 7, and 8), even though userNum, ruleNum, and SoDNum take relatively large values, it takes a short time to check the satisfaction of SoD constraints by PSoD-based, MER-based, or DSoD-based method. However, once the values of PermissionNum and RuleNum

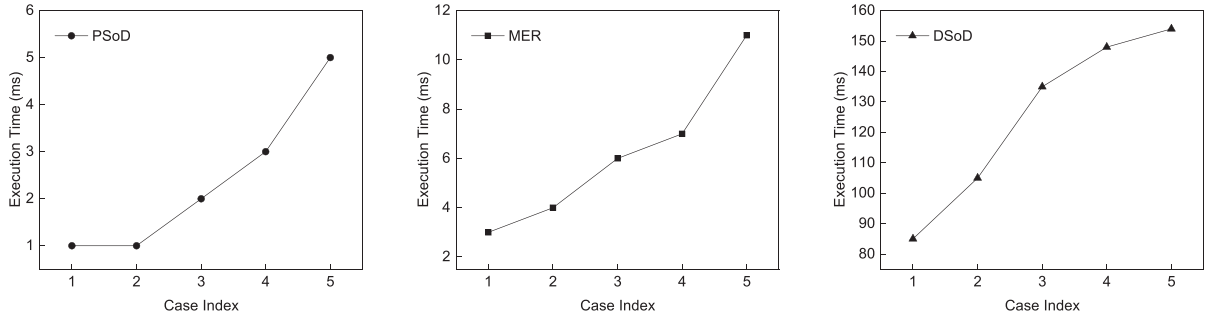


Fig. 6. Evaluate the effect of the increase in userNum, ruleNum, and SoDNum on the execution time required for checking SoD constraints based on PSOD-based method (in the left subfigure), MER-based method (in the middle subfigure), and DSoD-based method (in the right subfigure).

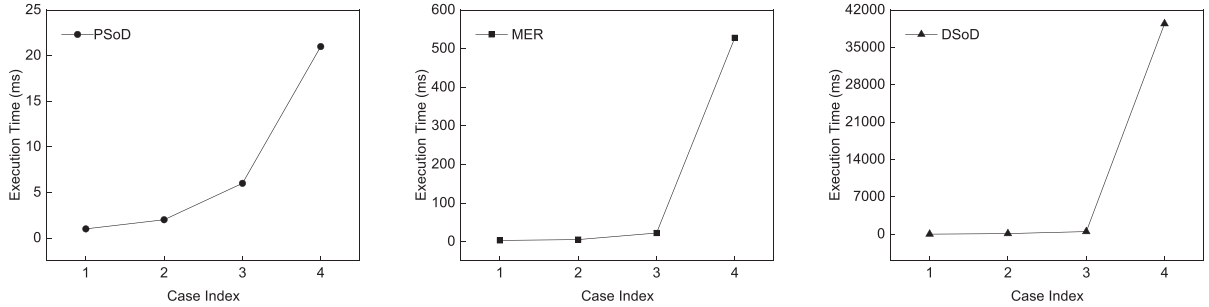


Fig. 7. Evaluate the effect of the increase in PermissionNum and RuleNum on the execution time required for checking SoD constraints based on PSOD-based method (in the left subfigure), MER-based method (in the middle subfigure), and DSoD-based method (in the right subfigure).

TABLE VIII
EVALUATE THE EFFECT OF PSOD-BASED, MER-BASED, AND DSoD-BASED METHODS FOR CHECKING THE SATISFIABILITY OF SoD CONSTRAINTS IN DIFFERENT CASES

No.	userNum	ruleNum	SoDNum	PermissionNum	RuleNum	PSOD time (ms)	MER time (ms)	DSoD time (ms)
1	50	50	20	3	3	4	10	207
2	50	50	20	5	5	6	18	371
3	50	50	20	10	10	15	o.o.m	o.o.m
4	100	100	30	3	3	10	18	215
5	100	100	30	5	5	16	38	874
6	100	100	30	10	10	30	o.o.m	o.o.m
7	500	500	100	3	3	220	284	5115
8	500	500	100	5	5	286	460	20803
9	500	500	100	10	10	741	o.o.m	o.o.m

o.o.m (the abbreviation of “out of memory”): the corresponding method cannot check the SoD constraints given in the case within a reasonable time due to a java.lang.OutOfMemoryError.

become large (Cases 3, 6, and 9), neither MER-based nor DSoD-based method can check the SoD constraints in the corresponding case within a reasonable time due to a java.lang.OutOfMemoryError (denoted as o.o.m). This happens because either MER-based or DSoD-based method is of exponential complexity with respect to PermissionNum and RuleNum, the total memory required to store the Cartesian product result of all relevant rule sets of all permissions for all SoD constraints increases exponentially with the increase of PermissionNum and RuleNum. When the values of PermissionNum and RuleNum are relatively large, the required memory space increases dramatically. This is beyond the memory available in the heap. As a result, a java.lang.OutOfMemoryError occurs. Nevertheless, thanks to the much lower complexity of PSOD-based method, the satisfaction checking results of all SoD constraints in each case can be obtained in a very short time.

Finally, we show the performance of PSOD-based method compared with MER-based and DSoD-based methods in different large-scale ABAC systems. The used SoD constraints include, but are not limited to, $\emptyset$ -SoD and $\blacktriangle$ -SoD constraints.

Table IX shows the performance of three methods on different cases. Here, we set the cardinality of each SoD constraint set as 2 or the number of permissions in the SoD constraint.

As shown in Table IX, for each test case, PSOD-based method can completely check the satisfaction of all SoD constraints specified in the case. The size of the large case is up to userNum = 5000, ruleNum = 5000, SoDNum = 150, PermissionNum = 100, and RuleNum = 200 (here, we choose the largest example with such a size because we believe this is sufficient to cover most common-seen and middle-sized systems and to demonstrate the theoretical achievements proposed in this paper. To be honest, it requires more effort to investigate the practical systems so as

TABLE IX
EVALUATE THE EFFECT OF PSoD-BASED, MER-BASED, AND DSoD-BASED METHODS ON DIFFERENT CASES WHERE THE CARDINALITY OF EACH SoD CONSTRAINT IS SET AS 2 OR THE TOTAL NUMBER OF PERMISSIONS IN THE SoD CONSTRAINTS

No.	userNum	ruleNum	SoDNum	PermissionNum	RuleNum	PSoD time (ms)	MER time (ms)	DSoD time (ms)
1	10	10	10	5	10	1	17	137
2	50	50	15	5	30	12	6105	13271
3	100	100	30	20	50	172	o.o.m	o.o.m
4	200	200	50	30	80	1041	o.o.m	o.o.m
5	500	500	80	50	100	14556	o.o.m	o.o.m
6	1000	1000	100	50	200	105958	o.o.m	o.o.m
7	5000	5000	150	100	200	7571547	o.o.m	o.o.m

o.o.m (the abbreviation of “out of memory”): the corresponding method cannot check the SoD constraints given in the case within a reasonable time due to a java.lang.OutOfMemoryError.

TABLE X
EVALUATE THE EFFECT OF PSoD-BASED AND DSoD-BASED METHODS ON DIFFERENT CASES WHERE THE CARDINALITY OF EACH SoD CONSTRAINT IS SET AS AN INTEGER WHICH IS NO MORE THAN THE TOTAL NUMBER OF PERMISSIONS IN THE SoD CONSTRAINTS

No.	userNum	ruleNum	SoDNum	PermissionNum	RuleNum	PSoD time (ms)	DSoD time (ms)
1	20	20	10	5	5	2	152
2	50	40	15	5	10	15	8640
3	100	80	25	10	50	72	o.o.m
4	300	150	50	30	80	1635	o.o.m
5	500	500	80	50	100	11003	o.o.m
6	1200	1000	110	80	150	162849	o.o.m
7	5500	5000	160	100	200	6922438	o.o.m

o.o.m (the abbreviation of “out of memory”): the corresponding method cannot check the SoD constraints given in the case within a reasonable time due to a java.lang.OutOfMemoryError.

to test the performance of this paper. We plan to do so in the near future). In contrast, either of MER-based and DSoD-based methods can only handle small cases (e.g., Cases 1 and 2), and for the relatively large ones, it fails to solve any case at all.

Table X reports the results of other broadly similar experiments while comparing PSoD-based method and DSoD-based method in different cases with general SoD constraints. That is, the cardinality of each SoD constraint can take an integer value that is not greater than the number of permissions in the SoD constraint.

From Table X, one can observe that DSoD-based method can only be used to check the satisfaction of SoD constraints in the first two small-scale cases, i.e., Cases 1 and 2. Consider that the running time of DSoD-based method grows exponentially with the sizes of the ABAC models and SoD constraints. Large systems (e.g., Cases 3–7) would require enormous computational power to be solved with DSoD-based method. It is not surprising that out of memory error occurs in a regular machine for Cases 3–7 when DSoD-based method is adopted.

Based on Table X, it can be seen that, even for the largest case with 5500 users, 5000 rules, 160 SoD constraints, 100 permissions in each SoD constraint, and 200 rules specified in each permission of an SoD constraint, it takes a feasible time for SoD verification by using PSoD-based method. This in turn implies that PSoD-based method significantly extends the range of solvable SC-SoD. Therefore, the proposed PSoD-based method is of practical significance for solving SoD verification problems in real-world ABAC systems.

VIII. CONCLUSION

This paper focuses on the satisfaction checking problem as well as violation (if exists) resolving problem of SoD constraints

in ABAC system. We show that by subdividing permissions into two classes so as to compute the minimal number of users to complete each class of permissions, respectively, the satisfiability of SoD constraints can be checked in an efficient way. We experimentally evaluate the performance of our approach compared with the existing ones. The experimental results show that the proposed approach performs efficiently and practically in large-scale ABAC systems. A 0–1 IP-based solution is given to solve SoD violations. This is useful to guarantee the authorization security of ABAC system.

In the future, we plan to investigate more complex ABAC systems and seek an efficient algorithm to solve SoD violations whose conflict-user permission set is empty. For example, we can assign each SoD violation with a cost, which can be regarded as an estimate of potential risk when allowing the violation. As a consequence, a solution to SoD violations can be obtained by adjusting access relations among users and rules with the minimal cost. In this paper, we consider the SoD constraints whose conflict-user permission set is not empty. There may exist SoD constraints whose conflict-user permission set can be empty. Therefore, we will analyze such SoDs in the future. Furthermore, it is interesting to check the satisfiability of other authorization constraints, such as binding of duty and constraints of cardinality.

REFERENCES

- [1] A. Armando and S. Ponta, “Model checking authorization requirements in business processes,” *Comput. Secur.*, vol. 40, pp. 1–22, Feb. 2014.
- [2] P. Bergé, J. Crampton, G. Gutin, and R. Watrigant, “The authorization policy existence problem,” *IEEE Trans. Dependable Secur. Comput.*, vol. 17, no. 6, pp. 1333–1344, Nov./Dec. 2020.
- [3] Y. Cheng, J. Park, and R. Sandhu, “An access control model for online social networks using user-to-user relationships,” *IEEE Trans. Dependable Secur. Comput.*, vol. 13, no. 4, pp. 424–436, Jul./Aug. 2016.

- [4] J. Crampton, "Specifying and enforcing constraints in role-based access control," in *Proc. 8th ACM Symp. Access Control Models Technol.*, Como, Italy, 2003, pp. 43–50.
- [5] Y. Du, Y. Wang, B. Yang, and H. Hu, "Analyzing security requirements in timed workflow processes," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 1, pp. 190–207, Jan./Feb. 2022.
- [6] G. Fedrechski, L. De Biase, P. Calcina-Ccori, R. de DeusLopes, and M. Zuffo, "SmartABAC: Enabling constrained IoT devices to make complex policy-based access control decisions," *IEEE Internet Things J.*, vol. 9, no. 7, pp. 5040–5050, Apr. 2022.
- [7] M. Gupta, R. Sandhu, T. Mawla, and J. Benson, "Reachability analysis for attributes in ABAC with group hierarchy," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 1, pp. 841–858, Jan./Feb. 2023.
- [8] M. Gupta, F. Awaysheh, J. Benson, M. Alazab, F. Patwa, and R. Sandhu, "An attribute-based access control for cloud enabled industrial smart vehicles," *IEEE Trans. Ind. Inform.*, vol. 17, no. 6, pp. 4288–4297, Jun. 2021.
- [9] E. Gupta, S. Sural, J. Vaidya, and V. Atluri, "Enabling attribute-based access control in NoSQL databases," *IEEE Trans. Emerg. Top. Comput.*, vol. 11, no. 1, pp. 208–223, Jan./Mar. 2023.
- [10] H. Huang and H. Kirchner, "Secure interoperation design in multi-domains environments based on colored Petri nets," *Inf. Sci.*, vol. 221, pp. 591–606, Feb. 2013.
- [11] V. Hu et al., "Guide to attribute based access control (ABAC) definition and considerations," *NIST Special Pub.*, vol. 800, no. 162, pp. 1–54, 2014.
- [12] H. Huang, J. Zhang, J. Hu, Y. Fu, and C. Qin, "Research on distributed dynamic trusted access control based on security subsystem," *IEEE Trans. Inf. Foren. Sec.*, vol. 17, pp. 3306–3320, Sep. 2022.
- [13] IBM, ILOG CPLEX Optimization Studio. Accessed: Oct. 5, 2022. [Online]. Available: <http://www-01.ibm.com/software/integration/optimization/cplexoptimization-studio/>
- [14] S. Jha, S. Sural, V. Atluri, and J. Vaidya, "Specification and verification of separation of duty constraints in attribute-based access control," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 4, pp. 897–911, Apr. 2018.
- [15] S. Jha, S. Sural, V. Atluri, and J. Vaidya, "Enforcing separation of duty in attribute based access control systems," in *Proc. Int. Conf. Inf. Syst. Secur.*, Kolkata, India, 2015, pp. 61–78.
- [16] T. Jaeger, "On the increasing importance of constraints," in *Proc. 4th ACM Workshop Role Access Control*, Fairfax, Virginia, USA, 1999, pp. 33–42.
- [17] X. Jin, R. Krishnan, and R. Sandhu, "A unified attribute-based access control model covering DAC, MAC and RBAC," in *Proc. 26th Annu. IFIP WG Conf. Data Appl. Secur. Privacy XXVI*, Paris, France, 2012, pp. 41–55.
- [18] H. Kim, D. Kim, and A. Alaerjan, "ABAC-based security model for DDS," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 5, pp. 3113–3124, Sep./Oct. 2022.
- [19] L. Karimi, M. Aldairi, J. Joshi, and M. Abdelhakim, "An automatic attribute-based access control policy extraction from access logs," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 4, pp. 2304–2317, Jul./Aug. 2022.
- [20] N. Li and Q. Wang, "Beyond separation of duty: An algebra for specifying high-level security policies," *J. ACM*, vol. 55, no. 3, pp. 12:1–12:46, Jul. 2008.
- [21] N. Li, M. Tripunitara, and Z. Bizri, "On mutually exclusive roles and separation-of-duty," *ACM Trans. Inf. Syst. Secur.*, vol. 10, no. 2, pp. 5:1–5:36, May 2007.
- [22] H. Nurmamat and K. Rahman, "Attribute based access control constraint based on subject similarity," in *Proc. IEEE Workshop Adv. Res. Technol. Ind. Appl.*, Ottawa, ON, Canada, 2014, pp. 226–229.
- [23] A. Roy, S. Sural, A. Majumdar, J. Vaidya, and V. Atluri, "Optimal employee recruitment in organizations under attribute-based access control," *ACM Trans. Manage. Inf. Syst.*, vol. 12, no. 1, pp. 6:1–6:24, Jan. 2021.
- [24] A. Roy, S. Sural, A. Majumdar, J. Vaidya, and V. Atluri, "Enabling workforce optimization in constrained attribute-based access control systems," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 4, pp. 1901–1913, Oct.–Dec. 2021.
- [25] R. Sandhu, E. Coyne, H. Feinstein, and C. Youma, "Role-based access control models," *IEEE Comput.*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [26] R. Sandhu and P. Samarati, "Access control: Principle and practice," *IEEE Commun. Mag.*, vol. 32, no. 9, pp. 40–48, Sep. 1994.
- [27] W. Susilo, P. Jiang, J. Lai, F. Guo, and R. Deng, "Sanitizable access control system for secure cloud storage against malicious data publishers," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 3, pp. 2138–2148, May/Jun. 2022.
- [28] B. Shafiq, J. Joshi, E. Bertino, and A. Ghafoor, "Secure interoperation in a multidomain environment employing RBAC policies," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 11, pp. 1557–1577, Nov. 2005.
- [29] Z. Wan, J. Liu, and R. Deng, "HASBE: A hierarchical attribute-based solution for flexible and scalable access control in cloud computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 7, no. 2, pp. 743–754, Apr. 2012.
- [30] K. Xue et al., "RAAC: Robust and auditable access control with multiple attribute authorities for public cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 4, pp. 953–967, Apr. 2017.
- [31] R. Xu, J. Joshi, and P. Krishnamurthy, "An integrated privacy preserving attribute-based access control framework supporting secure deduplication," *IEEE Trans. Dependable Secur. Comput.*, vol. 18, no. 2, pp. 706–721, Mar./Apr. 2021.
- [32] M. Xiao, H. Li, Q. Huang, S. Yu, and W. Susilo, "Attribute-based hierarchical access control with extendable policy," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1868–1883, 2022.
- [33] B. Yang and H. Hu, "Secure conflicts avoidance in multi-domain environments: A distributed approach," *IEEE Trans. Syst. Man, Cybern. Syst.*, vol. 51, no. 9, pp. 5478–5489, Sep. 2021.
- [34] B. Yang and H. Hu, "Dynamic implementation of security requirements in business processes," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 2, pp. 1352–1363, Mar./Apr. 2022.
- [35] B. Yang and H. Hu, "Analysis of authorization constraints via integer linear programming," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2258–2271, Mar. 2023.
- [36] B. Zaman, R. Krishnan, and R. Sandhu, "Towards an attributebased constraints specification language," in *Proc. IEEE Int. Conf. Soc. Comput.*, Alexandria, VA, USA, 2013, pp. 108–113.
- [37] B. Zaman, R. Krishnan, and R. Sandhu, "Constraints specification in attribute based access control," *Science*, vol. 2, no. 3, pp. 131–144, 2013.



Benyuan Yang received the BS degree in logistics engineering from Tianjin College, University of Science and Technology Beijing, Tianjin, China, in 2014, the MS degree in logistics engineering from University of Science and Technology Beijing, Beijing, China, in 2017, and the PhD degree in control theory and control engineering from Xidian University, Xi'an, Shaanxi, China, in 2022. He is currently a lecturer with Soochow University. His research interests include discrete event systems, business process management, and access control security. He has almost 20 publications in journals and conference proceedings in the above areas.



Hesuan Hu (Senior Member, IEEE) received the BS degree in computer engineering and the MS and PhD degrees in electromechanical engineering from Xidian University, Xi'an, China, in 2003, 2005, and 2010, respectively. He is currently a full professor with Xidian University and also a Collaborator with Nanyang Technological University, Singapore, and a researcher with Xi'an Jiaotong University, Xi'an. He is the inventor of more than 40 issued and filed patents in his fields of expertise. He has more than 170 publications in journals, book chapters, and conference proceedings in the areas of his research interests, which include discrete-event systems and their supervisory control techniques, Petri nets, automated manufacturing systems, multimedia streaming systems, autonomous vehicles, cyber security, and artificial intelligence. He was the recipient of many national and international awards, including the Franklin V. Taylor Memorial Award for Outstanding Papers from IEEE SMC Society in 2010 and the finalists of the Best Automation Paper from the IEEE ICRA Society in 2013, 2016, and 2017, respectively. He has been an associate editor for *IEEE Control Systems Magazine*, *IEEE Robotics and Automation Magazine*, *IEEE Transactions on Automation Science and Engineering*, *IEEE Robotics and Automation Letters*, and *Journal of Intelligent Manufacturing*. He is an IEEE Distinguished Lecturer.